ELO320 - Estructuras de Datos y Algoritmos
Introducción a C
Memoria Dinámica

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

2do Semestre 2022

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Memoria dinámica.

La memoria dinámica es aquella que cae bajo control del programador y se almacena el **heap** de la memoria.
Es un elemento necesario porque:

- Los arreglos fijos son espacios de memoria contigua cuyo tamaño no es modificable.
- ¿Qué pasa si definimos un tamaño y es insuficiente?
- ¿Qué pasa si el arreglo reserva más memoria que su uso real?

Memoria dinámica.

- Memoria Dinámica: Se reserva memoria en el heap.
- Se accede a esta memoria mediante punteros.
- 3 instrucciones básicas de `stdlib.h`:
  - `malloc()`: reserva de memoria.
  - `realloc()`: reasignación de memoria.
  - `free()`: liberación de memoria.
- `valgrind`: software para debugging de temas relacionados con memoria.

Memoria dinámica: `malloc`

- Instrucción para la reserva de memoria.
- Entrega como resultado la dirección de memoria donda inicia la reserva.
- **Necesitamos punteros**.
- Operador `sizeof` como base del tamaño a reservar.

C

```
#include<stdio.h>
#include<stdlib.h>

int main(int argc, char **argv){
        int *arreglo=NULL;

        arreglo = malloc(10 * sizeof(*arreglo));
//      arreglo = malloc(10 * sizeof(int));
//      arreglo = malloc(10 * sizeof *arreglo);

        // sizeof es un operador: sizeof(tipo_de_dato) o sizeof expresion
        // cast no es necesario, compilador hacecoerción.
        // solo para compatibilidad con C++, se usa
//      arreglo = (int *) malloc(10 * sizeof(int));
        return 0;
}
```

Memoria dinámica: realloc

- Instrucción para la reasignar memoria ya reservada.
- Si se agranda la memoria, esta puede cambiar completamente de dirección.

C

```c
#include<stdio.h>
#include<stdlib.h>

int main(int argc, char **argv){
        int *arreglo=NULL;

        arreglo = malloc(10 * sizeof(*arreglo));

        // libera 5 ints, mantiene dir.
        arreglo = realloc(arreglo, 5 * sizeof(int));

        // se expande en 15 ints, mantiene dir si existe
        // espacio contiguo, sino se cambia.
        arreglo= realloc(arreglo, 20 * sizeof * arreglo);

        return 0;
}
```

Memoria dinámica: free

- Instrucción para liberar la memoria reservada en heap.
- Los ejemplos anteriores no liberaban memoria.
- Sirve para liberar los recursos ociosos y no cometer en el manejo (memory leaks).

C

```c
#include<stdio.h>
#include<stdlib.h>

int main(int argc, char **argv){
        int *arreglo=NULL;

        arreglo = malloc(10 * sizeof(*arreglo));

        // libera 5 ints, mantiene dir.
        arreglo = realloc(arreglo, 5 * sizeof(int));

        // se expande en 15 ints, mantiene dir si existe
        // espacio contiguo, sino se cambia.
        arreglo= realloc(arreglo, 20 * sizeof * arreglo);

        free(arreglo);
        return 0;
}
```

Dangling

- **Dejar Colgado (dangling).**
- Puntero hacia localización de memoria del heap que ha sido liberada (e incluso nuevamente asignada).
- Genera errores poco predecibles (e.g. Segmentation fault, corrupción de memoria).

C

```
int main(int argc, char **argv){
        int x=2;
        int *p, *q;

        p= malloc(sizeof(int));
        q=p;

        free(p);  /* q aun mantiene referencia al heap */
                 /* puede que variable de heap sea reasignada */

        *q = x;

        return 0;
}
```

Fuga de memoria - Memory Leak

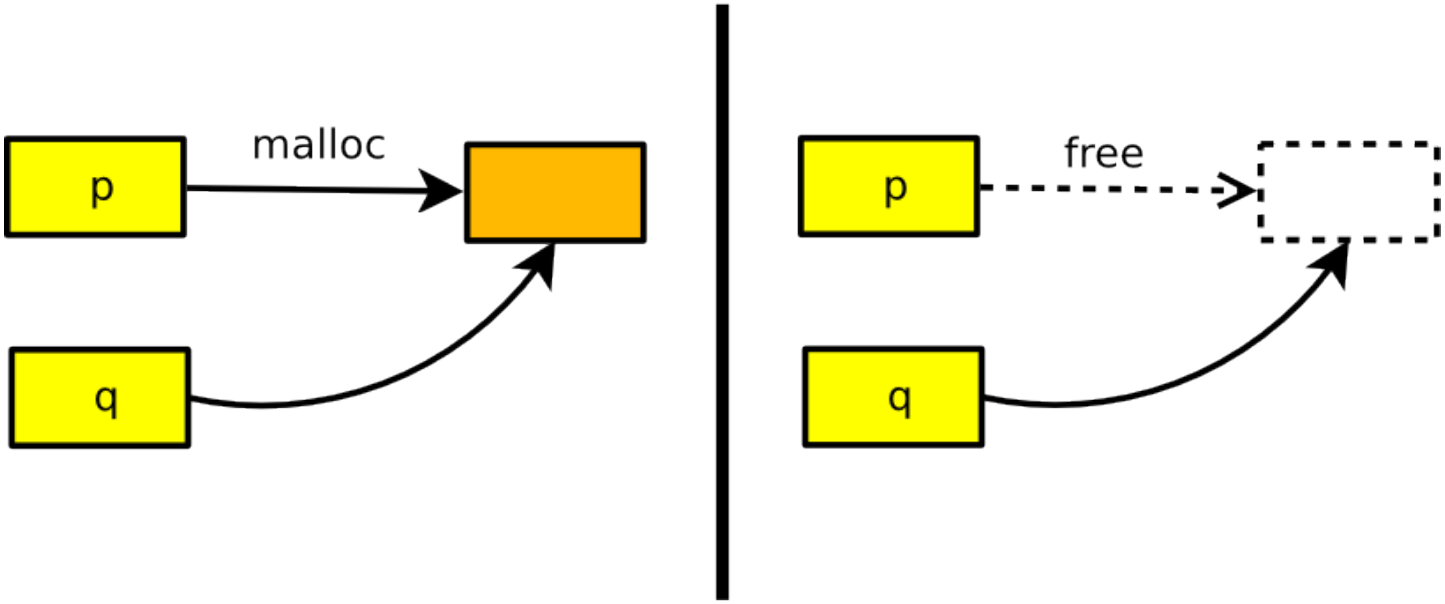
- **Fuga de memoria (memory leak)**.
- Pérdida de acceso a objeto de memoria reservado en el heap, por no existir variables que apunten a él.
- No se puede liberar hasta el término del programa.
- Puede saturar la memoria.

C

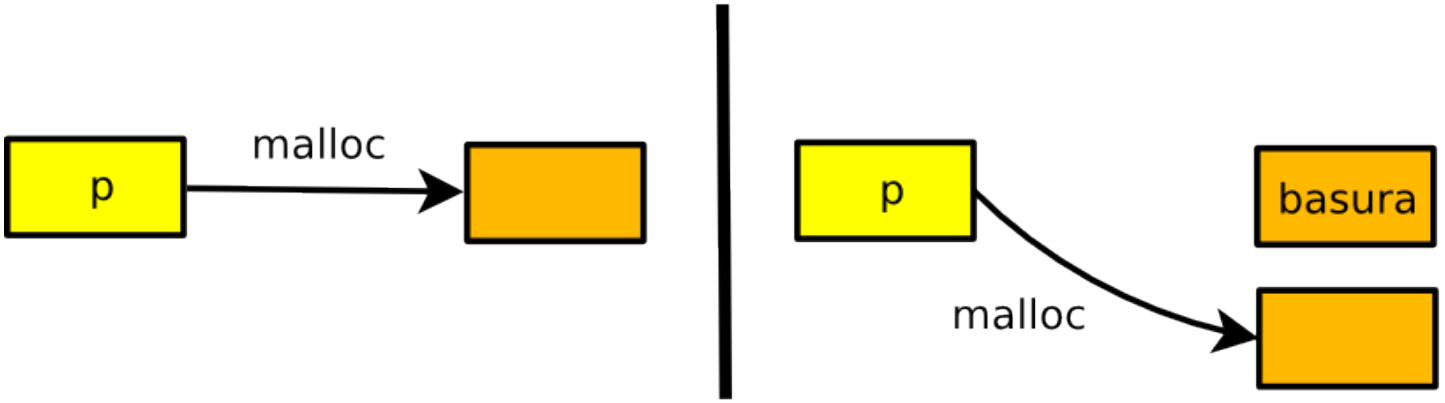
```
int main(int argc, char **argv){
        int *p;

        p= malloc(sizeof(int));
        p= malloc(sizeof(int));
        /* se pierde la direccion asignada anteriormente */

        free(p);

        return 0;
```

Ejemplo

Dangling

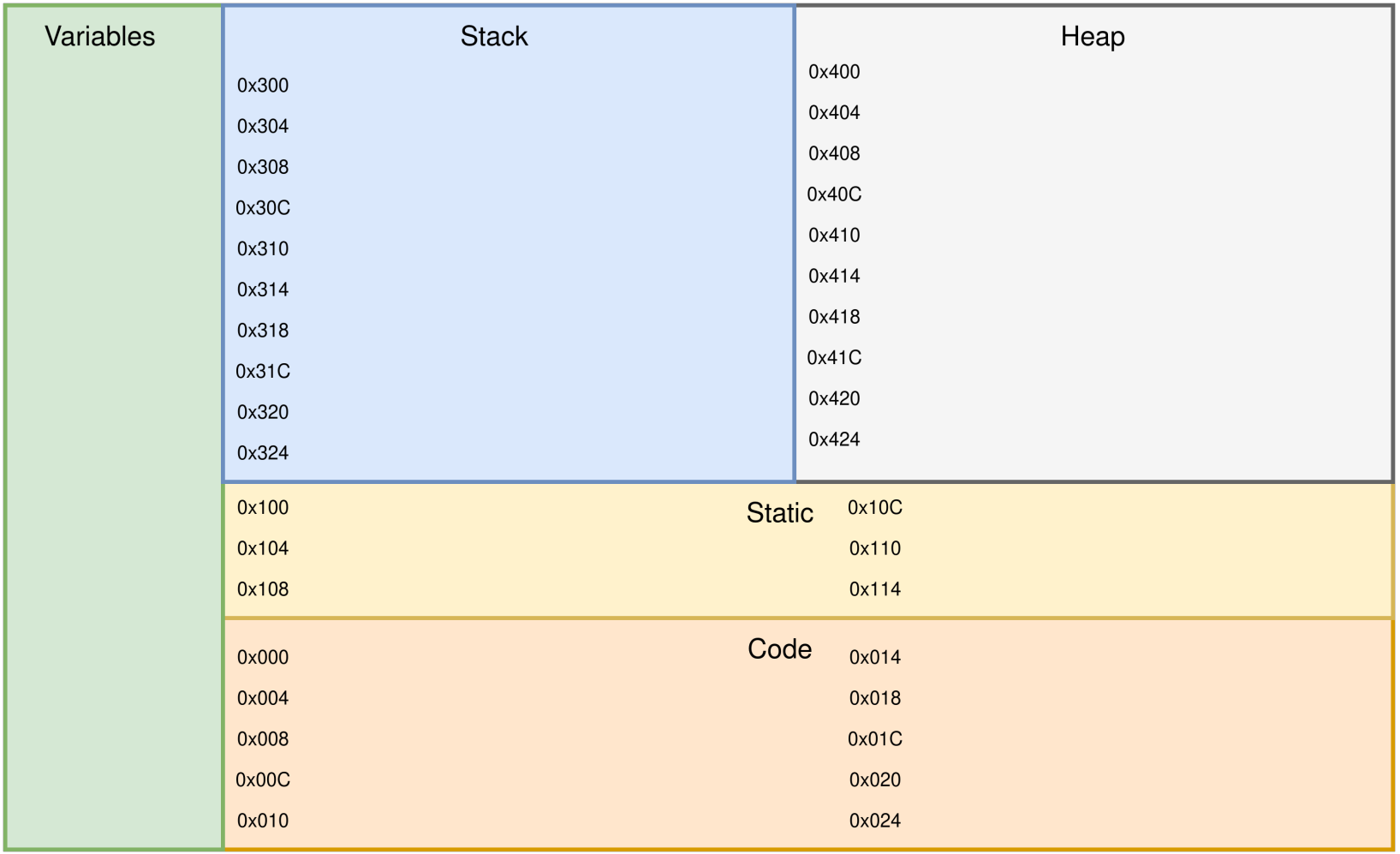


Memory Leak



Memoria dinámica: Ejemplo

C

```c
#include<stdio.h>
#include<stdlib.h>

int main(int argc, char **argv){

        int *arreglo=NULL, i;

        arreglo = malloc(10 * sizeof *arreglo); // sizeof(int) o sizeof(*arreglo)
        // sizeof es un operador: sizeof(tipo_de_dato) o sizeof expresion
        // cast no necesario, coerción. solo para compatibilidad con C++.

        if(arreglo==NULL){
                printf("Error, memoria mal reservada...\n");
                return -1;
        }

        for(i=0;i<5;i++){
                arreglo[i]=i+1;
        }

        arreglo = realloc(arreglo, 5* sizeof(int)); //reducimos a 5 ints

        for(i=0;i<5;i++){
                printf("%d ", arreglo[i]);
        }

        free(arreglo); //libera memoria
        return 0;
}
```

Memoria dinámica: Ejemplo